Concourse: The Event Loop

Concourse research notes

July 17, 2026

Contents

1 What this document is

`model_definition.tex` §8 queued four middle-layer questions. Design discussion on July 17, 2026 settled three and framed the fourth:

D1 (decided). `fire` emits enable/disable/re-enable deltas; the full `enabled(state)` recompute is retained as a debug-mode consistency check, not measured against — deltas are chosen for correctness of the order of operations, not for speed.

D2 (open; proposal in §3). The cascade worklist must settle to a fixed point; the policy that makes the fixed point unique is proposed here and awaits a decision.

D3 (decided). The interpreter owns the marked record. CompetingClocks samples, and only samples; its watchers observe sampling, and the marked record is not a sampling artifact.

D4 (decided). Sampler choice belongs to CompetingClocks' `SamplerBuilder`: Concourse describes the clock-key type and the distributions in play, and the builder picks. Any sampler valid for those distributions must work; none is load-bearing.

Charter claims F9–F11 (§8) extend the falsification charter of `model_definition.tex` §7.

2 D1: `fire` emits clock deltas

2.1 Why the diff loses

Beyond ordering, an enabled-set diff is *semantically blind to re-enablings*. When processor sharing re-evaluates every resident clock on an arrival, or a preemption changes which distribution a still-enabled key carries, the enabled set before and after the firing is identical — the diff sees nothing, while the sampler must receive `reenable!`. Deltas carry information the diff cannot express. The diff survives only as the debug oracle for the one thing it can check: membership.

2.2 The delta vocabulary

The contract's `fire` is θ-free and pure, so deltas cannot carry distributions — only keys and operations. The interpreter derives each distribution from `clock_distribution` at the post-firing state and each anchor from `st.te` (amendment A1):

```
const ClockDelta = Tuple{Symbol,ClockKey}   # (:enable | :disable | :reenable,
    key)
# fire returns (new_state, deltas::Vector{ClockDelta}); emission order is
# authoritative and is applied to the sampler verbatim.
```

This is ClockGradients' existing `fire_changes(model, state, key)` seam with the changes type made concrete (and the CD-1 time argument applied to it as well); `enabled_update` consumes the same deltas, so the estimators' incremental path and the interpreter share one vocabulary. The fired clock itself is implicit: the interpreter calls `fire!(ctx, key, t)` before applying deltas, so `fire` never emits a delta for its own key.

2.3 The debug oracle

After every firing, debug mode asserts two things:

1. *Membership:* `enabled(m, st')` equals the previous enabled set plus emitted `:enable`s minus emitted `:disable`s (and the fired key). This is the complete-recompute check, and it is the specification — `enabled` remains the meaning, deltas the implementation.

2. *Anchors:* the sampler's per-clock enabling times equal `st.te` — the A1 invariant. This is what checks re-enablings, which membership cannot see.

2.4 Refinement forced by F2: deltas are derived, not emitted

Running charter claim F2 (add reneging touching only the family registry and the surface constructor) falsified D1 *as first implemented*: patience clocks are born when a job enters a buffer and die when it leaves, and with deltas hand-emitted inside `settle!` and the family

`fire` functions, adding the family would have meant threading emission code through every state-transition site — interpreter surgery, which is exactly what F2 forbids. The repair keeps D1's decision and changes its mechanism:

- *Membership deltas are derived, not emitted.* The cascade already knows which stations it touched. After the cascade, for each touched station and each family, the family's per-station enabled keys are diffed between the old and new states (both exist; `fire` is pure). A family is now exactly four methods — `family_station_keys!`, `family_law`, `family_fire!`, `family_memory` — and clock lifecycle follows from membership alone.
- *Locality is preserved.* The user's original argument for deltas over a global diff was cost on large networks; the derived diff runs only over the touched stations' clocks, so the asymptotics stand. Explicit `:reenable` deltas remain available for the processor-sharing path (phase 2), which membership cannot see.
- *The A1 bookkeeping collapses into one place.* Enabling times and banked ages are updated in the same derivation pass, driven by the diffs and the per-family memory policy, instead of at every site that moves a job.
- *A correctness dividend.* A clock enabled and disabled within one firing (a job dispatched and immediately preempted in the same cascade) now nets out of the diff and never reaches the sampler — the GSMP has no zero-duration clocks, which the emission order previously only approximated by a disable/enable pair.

This is the falsification charter working as designed: the claim failed against the first implementation, and what survived is a better mechanism under the same decision. After the repair, the `:patience` addition itself changed only the registry and the surface fields, and M/M/1+M matches the Erlang-A birth–death form with the membership oracle running through every reneging cascade.

3 D2 proposal: the cascade worklist

3.1 The problem

Within one firing at time t, instantaneous consequences chain: a deposit triggers `settle!` at the destination; a departure from a full buffer lets a blocked upstream server transfer in, which frees that server, which admits from its buffer, which may unblock stations further upstream. The cascade must reach a fixed point — no station can dispatch, preempt, or unblock — before the sampler sees any delta. The requirement is that the fixed point be *unique and deterministic*, because `fire` is the replayed, differentiated object.

The GSPN literature has this exact problem as immediate transitions and vanishing markings, resolved there by priorities and weights (Zimmermann Ch. 5–6). The proposal below is that resolution, restated for queues.

3.2 Split contention from propagation

The order of cascade processing matters in exactly one place: when two consumers compete for one freed resource — two blocked upstream servers and one freed buffer slot; two idle servers and

one arriving job is already settled by the discipline's `select`. Everywhere else the steps merely propagate.

Contention is model semantics. Who gets a freed slot is an observable modeling choice, not an implementation detail, so it must be a declared policy value on the station, parallel to the discipline:

```
UnblockPolicy:
    FCFSUnblock()          # longest-blocked first (default)
    PriorityUnblock(by)    # by a ScalarExpr over the blocked job's marks
    RandomUnblock()        # a recorded choice, via the A4 draw source
```

`FCFSUnblock` needs the blocking order, so the blocked-server set in `QueueState` is an ordered vector, not a set. `RandomUnblock` consumes a recorded draw and so stays inside the A4 randomness census.

Propagation must be order-free. With contention delegated to policy, the claim is that the remaining cascade relation is *confluent*: any fair processing order reaches the same fixed point. Termination comes from check C3 (acyclic blocking chains): each unblocking transfer moves a job strictly downstream in the blocking DAG, so the cascade measure is well-founded. Confluence plus termination give a unique normal form (Newman's lemma); the worklist order is then an implementation choice, not a semantic one.

The canonical order, so that the implementation is deterministic even if the confluence claim fails: a FIFO worklist of stations to settle, seeded with the destination station then the origin station of the firing — the one-hop order `queue_layers.tex` §3.2 fixed by fiat, generalized. A debug-build fuel counter bounds the cascade at (jobs + servers) steps and turns nontermination into an error rather than a hang.

3.3 What falsifies it

F9 in the charter: run the same trajectories with FIFO and LIFO worklists; `states_equal` at every index or the confluence claim is dead, and the worklist order gets promoted into the IR's semantics the way the one-hop order already is. Either outcome is a result; the test is cheap.

4 D3: the interpreter owns the record

The marked record (k_i, t_i, a_i) is the primary observable of the whole library, and its auxiliary-draw column exists because the *interpreter* supplies the draw source (A4) — the sampler never sees those draws, so a sampler-side watcher could never capture them. The interpreter appends one entry per firing: the key, the time, and the draws consumed by that firing in consumption order. CompetingClocks' watchers remain what they are — instrumentation of sampling — and Concourse attaches none of the record path to them. `gradient_record` gets one method, from the marked record type, per `model_definition.tex` §6.

One replay detail follows from record ownership: during estimation, `fire` must read firing i's recorded draws, and the contract does not pass i. `QueueState` therefore carries a firing counter (`nfired::Int`, incremented by every `fire`), and the replay binding indexes the record with it. A pure counter in a pure state; the same device as `next_id`.

5 D4: the builder chooses the sampler

Concourse tells `SamplerBuilder` the clock-key type and lets CompetingClocks choose the sampler for the distributions in play; the default is whatever the builder decides, and Concourse hard-codes no method. The design consequence is a claim, not a configuration: *every* sampler valid for the model's distribution set must produce statistically identical results, because the interpreter speaks only the blessed `SamplingContext` surface (`enable!`, `disable!`, `reenable!`, `next`, `fire!`) and no model code reaches around it. Estimator-driven overrides (the carry coupling for IPA) are passed through as builder arguments by the estimation entry points, not chosen by the model.

6 The interpreter, whole

With D1–D4 fixed, the middle layer is small enough to state completely. Julia-flavored pseudocode; error handling and the debug oracle elided:

```
function simulate(m::QueueGSMP, θ, horizon, seed)
    st  = initial_state(m)
    ctx = SamplingContext(build(m), seed)     # D4: builder chooses
    rec = MarkedRecord(clockkeytype(m))
    for k in enabled(m, st)                   # initial enablings
        enable!(ctx, k, clock_distribution(m, θ, k, st), st.te[k], 0.0)
    end
    while true
        (t, key) = next(ctx, st.time)
        (key === nothing || t > horizon) && break
        fire!(ctx, key, t)                    # realize the winner's draw
        draws = draw_source(ctx, key)         # A4: keyed, recording
        st, deltas = fire_changes(m, st, key, t, draws)   # cascade inside
        push!(rec, key, t, consumed(draws))   # D3: interpreter-owned
        for (op, k) in deltas                 # D1: emission order verbatim
            op === :disable  && disable!(ctx, k, t)
            op === :enable   && enable!(ctx, k, clock_distribution(m, θ, k, st),
    st.te[k], t)
            op === :reenable && reenable!(ctx, k, clock_distribution(m, θ, k,
    st), st.te[k], t)
        end
    end
    close_record!(rec, min(horizon, st.time))
end
```

Everything model-specific is inside `fire_changes` (families, disciplines, kernels, the cascade); everything stochastic is inside `ctx` and `draws`; everything observable is `rec`. The loop itself should never grow.

7 Phase 2: the processor-sharing compilation (F8)

Implemented July 17, 2026. Three findings, one of them a deliberate departure from `queue_layers.tex`.

The anchor convention. `queue_layers.tex` §3.5 compiled a speed change as a re-enabling *anchored at the current time*, with the banked age moved into a left truncation and an explicit

warning against also keeping the original enabling time. Concourse does the opposite: `te` stays at the original enabling forever, and a speed change is a *mid-flight re-evaluation* of the clock's wall-time law

$$\text{age} \;\sim\; \text{shift} + (X - a)/r \;\Big|\; X > a, \qquad X \sim F,\ \text{shift} = \text{anchor} - t_e,$$

carried by the state's `bank` (internal age a at the last change) and `anchor` (that change's wall time) — an A1 addendum. The reason is the ClockGradients contract: the Bookkeeper reconstructs `te` as the enabling firing's time, the score replay accumulates piecewise survival against distributions anchored at that fixed `te`, and the segment machinery (CG-M3) exists precisely to represent re-evaluations at constant anchor. Under the anchor-at-now convention every speed change would invalidate the reconstructed `te` and the whole estimation stack; under the fixed-anchor convention it composes with no changes at all. The sampler side is the age-keeping `reenable!` (`relative_te = te − now` ≤ 0), which CompetingClocks documents as exactly this case. The interpreter did not change: the `:reenable` branch it already had for D1 computes that shift.

The re-evaluation hook. The derived-delta pass (§2.4) gained its promised explicit-`:reenable` seam: `family_reenables!`, a per-family hook run after the membership diff at each touched station. The `:service` method detects a resident-count change at a PS station, banks each survivor's internal age at the old speed, moves its anchor, and emits `(:reenable, k)`. A `:rescale` memory mode joins `:fresh`/`:resume`: enabling anchors `te` at now and clears the bank, and speed bookkeeping otherwise stays out of the generic passes.

Dual-safety forced a bespoke distribution. Composing `Distributions.truncated` and affine wrappers builds the right law but poisons ForwardDiff: `Truncated`'s stored bound constants produce NaN partials in `logccdf` (and a lower bound of zero stores $\log 0 = -\infty$ with NaN partials outright). `SharedRemaining` in `src/semantics.jl` implements the conditional-remaining law directly — every method a two-call delegation to the base distribution — and is the object the sampler draws from, the likelihood differentiates, and the Bookkeeper compares for segment detection. (The hunt also surfaced a one-line short-circuit-order bug in CompetingClocks' `lefttrunc.jl`, fixed there.)

The oracles: M/M/1-PS matches $\rho/(1-\rho)$; M/G/1-PS with Gamma service is *insensitive* (same mean, while the same law under FCFS matches its distinct Pollaczek–Khinchine mean — proving the discipline actually shares); replay reproduces `bank`/`anchor` exactly; every resident-count change emits re-enables for every survivor; and the score gradient matches finite differences through genuine segment chains — the first exercise of CG-M3's mid-flight machinery by a real model, and the validation of the capability table's "likelihood ✓" row for PS.

8 Charter extensions

Continuing the numbering of `model_definition.tex` §7:

F9 (cascade confluence). With contention resolved by `UnblockPolicy`, the cascade fixed point is order-independent. *Test:* FIFO vs. LIFO worklists over random finite-buffer `:block` networks; `states_equal` at every firing index. Falsified ⇒ the worklist order becomes IR semantics.

F10 (sampler agnosticism). No charter result depends on the sampler. *Test:* run the F1–F7 oracle comparisons under at least two builder outcomes (e.g. first-to-fire and next-reaction); all pass under both.

F11 (delta fidelity). Emitted deltas agree with the recompute oracle. *Test:* debug-mode membership and anchor assertions (§2) hold over long mixed runs including preemption — and, in phase 2, processor sharing, which is the case only the anchor check can see.